

Lab 4 Report — Layla as a CrewAI Crew

1. Executive Summary

This lab evaluates the re-implementation of the Layla support agent using the **CrewAI** framework compared to a hand-rolled baseline agent. The evaluation focuses on structural control, gate enforcement, module state isolation, and prompt generation overhead.

2. Core Lab Questions & Technical Analysis

Q1: Where did the gates go? What does a new tool inherit?

In the CrewAI framework, safety and policy gates (such as pattern verification, order existence checks, and evidence requirements) are moved directly into the **tool bodies** (e.g., inside `track_order` and `request_approval`). CrewAI does not provide a centralized execution layer (like a custom dispatcher) to enforce pre-call policies.

When creating a new tool, it inherits only the JSON schema generated from Python type hints and the docstring description. It **does not inherit any safety gates or business logic rules**. Every new tool added to the system requires manual, scattered implementation of its own checks, making system-wide auditing difficult.

Q2: What is SEEN? What happens with two concurrent requests?

SEEN is a module-level global set used to track which tools have been executed during a run (e.g., ensuring `track_order` and `get_policy` were executed before requesting a credit approval). Because CrewAI lacks a dedicated request-scoped context container, SEEN acts as shared module state. In a multi-threaded or concurrent server environment handling two user requests simultaneously, both requests would read and write to the same SEEN variable. This creates a severe **state pollution and data leakage security vulnerability**, where one request could pass approval gates based on evidence gathered by a completely different user's request.

Q3: Analysis of Generated Schemas (prompt output)

Running the prompt command reveals the system prompts and tool schemas dynamically generated by CrewAI. The formal pattern constraint `^A[0-9]{4}$` is entirely absent from the generated tool argument schemas.

Type hints carry structural types (e.g., `str`, `int`), but they cannot carry business rules or validation judgements. Consequently, input boundary enforcement relies entirely on defensive checks embedded within the tool code rather than LLM-side schema constraints.

3. Gates Execution Output (No Model Required)

The following trace confirms that all safety gates function deterministically at the tool level without relying on an LLM:

Gates, with no model involved:

```
gate 3 fabricated id      {"ok": false, "error": "unknown_order",  
"hint": "Ask the customer to confirm the ID."}  
gate 2 bad pattern       {"ok": false, "error": "bad_pattern",  
"hint": "Order IDs look like A1032."}  
gate 4 no evidence yet   {"ok": false, "error": "no_evidence",  
"hint": "Check the order and the policy first."}  
gate 4 below threshold   {"ok": false, "error": "below_threshold",  
"hint": "Policy requires 10+ days."}  
all gates pass           {"ok": true, "approval_ref": "APR-2048",  
"state": "pending", "account_changed": false}
```

Note the last one: `account_changed=false`. It proposed; a human disposes.